

Precise “Sensorless” Telescope Mount Homing

David Fobar
david_fobar@mines.edu

Abstract—Equatorial telescope mounts used in astrophotography require precise knowledge of rotational position to enable periodic error correction (PEC) of gear train imperfections. Mounts with harmonic drives exhibit periodic positional errors of 10–40 arcseconds at the output, rendering long exposures unusable without correction. PEC requires a repeatable home position, yet many mounts lack the homing switches or high-resolution absolute encoders necessary to establish one. We present an eight-state Extended Kalman Filter that jointly estimates the rotational positions of both the Right Ascension and Declination axes and accelerometer bias, using only the gravity projections measured by two body-fixed accelerometers during a controlled slew. Joint estimation of both axes is required, as neither axis alone provides sufficient information to unambiguously resolve the home state. The filter accurately homes the mount 99% of simulated iterations – dependent on the site latitude and limited by the observability of the RA axis.

I. INTRODUCTION

Astrophotography with amateur equatorial mounts demands precise mechanical repeatability that consumer hardware rarely delivers. A motorized equatorial mount tracks celestial objects by rotating the Right Ascension (RA) axis at the sidereal rate – approximately 15° per hour – while the Declination (DEC) axis remains stationary. Any deviation from ideal motion causes stars to drift across the image sensor, limiting practical exposure time.

The mount described in this work uses a NEMA 17 stepper motor on each axis, coupled through a 50:1 planetary gearbox in series with a 100:1 harmonic drive, yielding an effective gear ratio of 5000:1. With 200 full steps per revolution of the stepper motor and 256x microsteps per step, the commanded angular resolution is approximately 5.06 milli-arcseconds per microstep. However, the cyclic nature of the compound gear train introduces *periodic error*: a systematic, repeatable position deviation that completes one cycle per revolution of the worm or drive gear. For harmonic drives, this error is typically 10–40 arcseconds at the mount output. With the imaging setup used here – a Nikon Z6 paired with a 1377mm focal length telescope, where each pixel subtends approximately 0.89 arcseconds – uncorrected periodic error produces up to 0.4 arcsec/sec of uncorrected motion, causing measurable star trailing in exposures beyond 1–2 seconds.

Periodic Error Correction (PEC) mitigates this by precomputing a correction table that maps each gear-train phase to a compensating velocity offset. During sidereal tracking, the mount reads the current gear phase and applies the corresponding correction, effectively canceling the systematic drift. Because the error is highly repeatable, a single calibration measurement – typically acquired by guiding on a star and recording the residual motion over one or more complete gear

cycles – can produce a correction table that remains valid indefinitely. Critically, PEC is only effective if the mount can reliably identify its gear phase at startup: the correction table is indexed by position within the periodic error cycle, so an incorrect phase assignment shifts the correction out of alignment with the actual error, potentially worsening tracking rather than improving it. This requires a homing routine capable of locating the mount within 180° of the stepper input shaft – corresponding to $\pm 25,600$ microsteps – at which point a low-cost rotary encoder on the input shaft can slew to a precise, repeatable shaft position and establish a known gear-train phase.



(a) Mount overview.

(b) Rotational axes.

Fig. 1: The custom telescope mount.

The mount provides no absolute encoders or homing limit switches, but each rotational axis carries a rigidly mounted accelerometer with its z -axis aligned to the axis of rotation. During a controlled slew, each accelerometer measures the projection of the gravity vector onto its body frame, which is a deterministic function of the axis angle and several unknown constant parameters (accelerometer bias and sensor mounting misalignment). This paper presents a two-pass Extended Kalman Filter (EKF) that exploits these measurements to estimate the home position of both axes. In Pass 1, the filter simultaneously estimates the six accelerometer bias states and the two axis angles. The two additional constant mounting offsets exist, however attempting to estimate those as well provides an unobservable system (discussed in Section IV), and their offset can be absorbed into the homing position without affecting homing accuracy. Pass 2 freezes the constant bias parameters at their Pass 1 estimates and runs a reduced 2-state filter over the same measurement sequence initializing

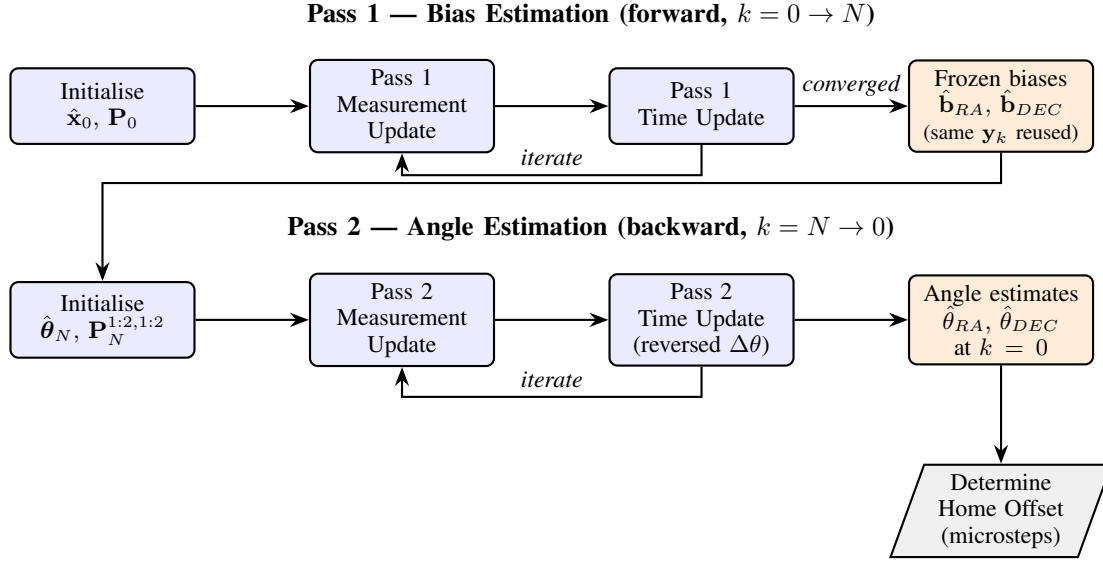


Fig. 2: Two-pass EKF architecture. Pass 1 sweeps forward over the measurement sequence to jointly estimate accelerometer biases and axis angles. Once converged, the bias estimates are frozen and Pass 2 sweeps the same sequence in reverse as a reduced 2-state filter, converging angle uncertainty toward $k = 0$ where the home position is extracted.

the angle estimates with the final state of Pass 1. Pass 2 yields estimates of θ_{RA} and θ_{DEC} with high precision, often within the 25,600 microstep homing budget. Joint estimation of both axes is essential: neither axis provides sufficient information to resolve the home state independently, as the DEC accelerometer measurements are a function of both θ_{RA} and θ_{DEC} through the coupled rotation chain.

A. Estimation Scenario

At startup, the precise angular position of each axis is unknown, though multiple samples from the accelerometers can provide an estimate within 2–3 degrees to initialize the Pass 1 filter. The mount performs a controlled slew, driving each axis at a commanded constant angular velocity. Because the gear ratio and step resolution are precisely known, the commanded velocity is treated as a deterministic input. The step pulses are generated by dedicated microprocessor pulse-generation peripherals with negligible time variance. However, the accelerometer is sampled by the main microprocessor loop, which introduces timing jitter in the relationship between the recorded step position and the accelerometer reading, modeled as additive Gaussian noise on the step position input to the filter.

B. Measurement Model

Accelerometer measurements are taken during the slew and related to the gravity vector projected onto each sensor's body frame. Gravity is first expressed in the polar-aligned mount frame by rotating through the site latitude, λ :

$$\mathbf{g}_p = \mathbf{R}_{lat} \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} = \begin{bmatrix} g \cos \lambda \\ 0 \\ -g \sin \lambda \end{bmatrix}$$

The RA accelerometer is fixed to the RA stage and rotates with it, so its measurements depend only on θ_{RA} . The DEC accelerometer is mounted to the DEC stage, which is itself rotated by the RA stage, so its measurements depend on both θ_{RA} and θ_{DEC} . Each sensor has a fixed nominal mounting orientation relative to its rotation axis, described by $\mathbf{T}$, and a constant zero- g bias $\mathbf{b}$. The combined measurement equations are:

$$\mathbf{h}_{RA}(\theta_{RA}) = \mathbf{R}_{RA}(\theta_{RA})\mathbf{T}_{RA}\mathbf{g}_p + \mathbf{b}_{RA} \quad (1)$$

$$\begin{aligned} \mathbf{h}_{DEC}(\theta_{RA}, \theta_{DEC}) &= \mathbf{R}_{DEC}(\theta_{DEC})\mathbf{T}_{DEC} \\ &\quad \times \mathbf{R}_{RA}(\theta_{RA})\mathbf{T}_{RA}\mathbf{g}_p + \mathbf{b}_{DEC} \end{aligned} \quad (2)$$

where the axis rotation matrices are:

$$\mathbf{R}_{RA}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3)$$

$$\mathbf{R}_{DEC}(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} \quad (4)$$

The RA home position is defined by $h_{RA,y} = 0$, and the DEC home position by $h_{DEC,z} = 0$ when RA is simultaneously at its home position. Neither condition alone is sufficient: $h_{DEC,z} = 0$ can occur at any RA angle, so joint estimation of both axes is required to unambiguously resolve the home state.

II. EXPERIMENTAL CHARACTERIZATION

Prior to simulation, experimental data was collected from the physical mount to characterize the system and validate the measurement model. Both axes were driven simultaneously through a homing sweep while accelerometer data was

recorded at approximately 100 Hz by the main processing loop. The RA axis was swept through approximately 115° of arc while the DEC axis was swept through approximately 90° of arc. The step interval statistics confirm that timing jitter is a property of the accelerometer sampling loop rather than the step pulse generation. The RA and DEC axes show mean intervals of 4923 and 4903 microsteps respectively, each with a standard deviation of approximately 1200 microsteps – a coefficient of variation of approximately 25% – which is used directly as σ_{step} in the simulation.

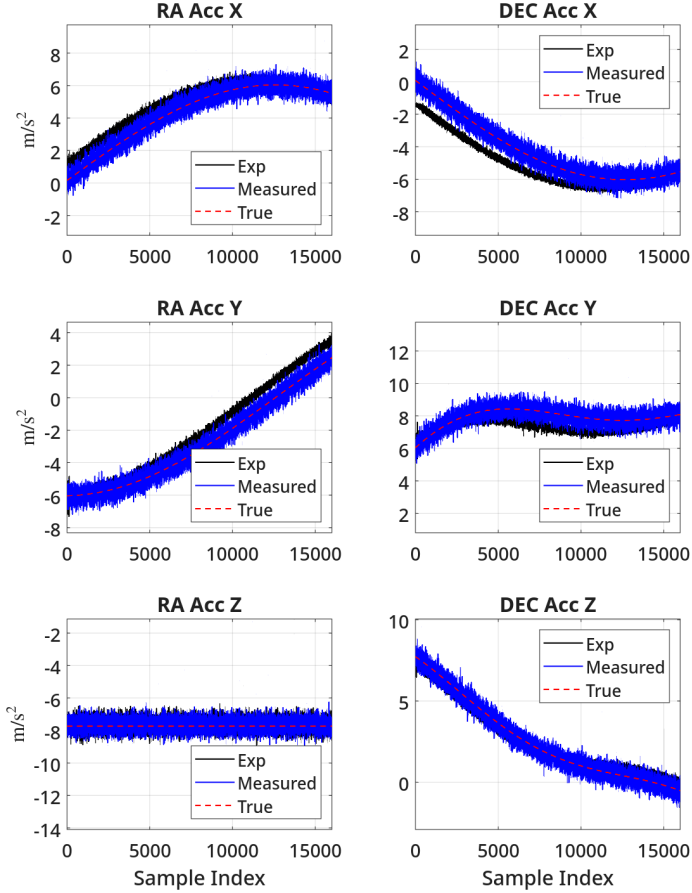


Fig. 3: Experimental (black) and simulated (blue) accelerometer measurements with true noiseless trajectories (red dashed) for simultaneous RA and DEC sweeps. Close agreement in signal amplitude and noise characteristics validates the simulation parameters. The coupling between axes is evident in the DEC measurements, which vary as a function of both θ_{RA} and θ_{DEC} through the rotation chain defined in the introduction.

The RA sweep shows the expected sinusoidal gravity projection onto the X and Y axes as the mount rotates, with the Y-axis crossing zero near the home position. The Z-axis remains approximately constant at $-g \sin \lambda \approx -7.73 \text{ m/s}^2$, consistent with the polar-aligned mount geometry at site latitude $\lambda = 52^\circ$, matching the location at which experimental data was collected (West Point, New York). This Z-axis, which is insensitive to rotation, provides a clean estimate of the accelerometer noise floor ($\sigma_{acc} \approx 0.34 \text{ m/s}^2$). The DEC sweep shows the Z-axis

varying sinusoidally from $+g \sin \lambda$ toward zero as the axis approaches home, while the Y-axis remains nearly constant – consistent with the physical mounting geometry in which the DEC rotation axis is aligned with the negative X-axis of the RA sensor frame.

A simulation of the estimation scenario was developed in MATLAB with parameters derived directly from experimental characterization. Accelerometer sampling jitter is modeled as additive Gaussian noise on the step position with standard deviation of 25% of the mean step interval. Constant accelerometer biases are sampled uniformly within $\pm 0.1 \text{ m/s}^2$ per axis. Measurement noise is modeled as zero-mean Gaussian with standard deviation $\sigma_{acc} = 0.34 \text{ m/s}^2$, matched to the experimental noise floor. While the experimental sweep covered approximately 115° and 90° of arc for RA and DEC respectively, the simulation extends both axes to a full 180° sweep to ensure sufficient trajectory coverage for filter convergence.

III. SYSTEM STATE SPACE MODEL

The full state vector is provided by:

$$\mathbf{x} = \begin{bmatrix} \theta_{RA} \\ \theta_{DEC} \\ \mathbf{b}_{RA} \\ \mathbf{b}_{DEC} \end{bmatrix} \in \mathbb{R}^8 \quad (5)$$

where $\mathbf{b}_{RA}, \mathbf{b}_{DEC} \in \mathbb{R}^3$ are the constant bias vectors for each accelerometer.

Since no motion takes place unless an input is provided, $\mathbf{A} = \mathbf{0}$. The controller input $\mathbf{u}(t) = [\omega_{RA}, \omega_{DEC}]^\top$ takes the form of angular velocity commands, which are decoupled between axes. Therefore the state equation is:

$$\dot{\mathbf{x}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{u}(t) + \mathbf{w}(t) \in \mathbb{R}^8 \quad (6)$$

where $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$. The process noise covariance $\mathbf{Q}$ reflects the timing jitter between the recorded step position and the accelerometer sample, and small random walk noise on the bias states.

$$\mathbf{Q} = \begin{bmatrix} \sigma_{step}^2 & 0 & \mathbf{0} & \mathbf{0} \\ 0 & \sigma_{step}^2 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \sigma_{bias}^2 \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \sigma_{bias}^2 \mathbf{I}_3 \end{bmatrix} \in \mathbb{R}^{8 \times 8} \quad (7)$$

The measurement equation is:

$$\mathbf{h}(\mathbf{x}) = \begin{bmatrix} \mathbf{h}_{RA}(x_1) \\ \mathbf{h}_{DEC}(x_1, x_2) \end{bmatrix} \quad (8)$$

$$\mathbf{y}(t) = \mathbf{h}(\mathbf{x}) + \mathbf{n}(t) \in \mathbb{R}^6 \quad (9)$$

where $\mathbf{h}_{RA}$ and $\mathbf{h}_{DEC}$ are the nonlinear measurement functions defined in Section I, $x_1 = \theta_{RA}$, $x_2 = \theta_{DEC}$, $\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ with $\mathbf{R} = \sigma_{acc}^2 \mathbf{I}_6$.

IV. OBSERVABILITY ANALYSIS

Before applying the EKF, the observability of the eight-state system is assessed to confirm that all states can be uniquely recovered from the accelerometer measurements along the planned trajectory. A discrete-time state space system is observable if and only if the observability matrix has full column rank. Since the process model is linear with $\Phi = \mathbf{I}_8$, the observability matrix reduces to the stacked measurement Jacobian over the trajectory. For the nonlinear system considered here, $\mathbf{C}_k$ varies with the state estimate at each step, with the analytical form derived in Section V-A.

$$\mathcal{O} = \begin{bmatrix} \mathbf{C} \\ \mathbf{C}\Phi \\ \vdots \\ \mathbf{C}\Phi^{n-1} \end{bmatrix} \xrightarrow{\Phi=\mathbf{I}} \begin{bmatrix} \mathbf{C}_1 \\ \mathbf{C}_2 \\ \vdots \\ \mathbf{C}_N \end{bmatrix} \quad (10)$$

The Observability matrix computed over the planned trajectory has full rank 8, confirming that all states are uniquely recoverable and the system is observable. The eigendecomposition is given in Table I. The smallest eigenvalue corresponds to $b_{DEC,z}$, consistent with the lagging convergence observed in Figure 4. Among the angle states, $\lambda_8/\lambda_7 \approx 2.3$, indicating that θ_{RA} is less observable than θ_{DEC} over the planned trajectory, therefore it is expected for the RA axis to be the limiting factor in homing performance.

TABLE I: Observability Gramian eigenvalues (ascending).

i	Dominant state	λ_i
1	$b_{DEC,z}$	9.68×10^3
2	$b_{RA,y}$	3.87×10^4
3-6	$b_{RA,x}, b_{RA,z}, b_{DEC,x}, b_{DEC,y}$	5.12×10^4
7	θ_{RA}	2.36×10^6
8	θ_{DEC}	5.43×10^6

Two additional mounting offset states were considered during system design but excluded from the state vector. Including them causes $\mathcal{O}$ to lose full column rank – their effect on the measurements is indistinguishable from a shift in the home angle – so the system becomes unobservable. Their contribution is instead absorbed into the homing position estimate without loss of generality.

V. EXTENDED KALMAN FILTER IMPLEMENTATION

The nonlinear measurement functions $\mathbf{h}_{RA}$ and $\mathbf{h}_{DEC}$ preclude the use of a standard Kalman Filter. As a result an EKF is employed, detailed in this section. This leverages the analytical Jacobian of the measurement model to linearize about the current state estimate at each step.

A. Analytical Jacobian Derivation

The EKF requires the Jacobian $\mathbf{C}_k = \partial \mathbf{h} / \partial \mathbf{x} |_{\hat{\mathbf{x}}_k}$ of the stacked measurement vector with respect to the state vector defined in (5). The partial derivatives of $\mathbf{R}_{RA}$ and $\mathbf{R}_{DEC}$ with respect to their respective angles are

$$\frac{\partial \mathbf{R}_{RA}}{\partial \theta_{RA}} = \begin{bmatrix} -\sin \theta_{RA} & -\cos \theta_{RA} & 0 \\ \cos \theta_{RA} & -\sin \theta_{RA} & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\frac{\partial \mathbf{R}_{DEC}}{\partial \theta_{DEC}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -\sin \theta_{DEC} & -\cos \theta_{DEC} \\ 0 & \cos \theta_{DEC} & -\sin \theta_{DEC} \end{bmatrix}$$

From (1), $\mathbf{h}_{RA}$ depends on θ_{RA} alone through $\mathbf{R}_{RA}$, and the bias terms enter additively, giving

$$\frac{\partial \mathbf{h}_{RA}}{\partial \theta_{RA}} = \mathbf{T}_{RA} \dot{\mathbf{R}}_{RA} \mathbf{g}_p$$

$$\frac{\partial \mathbf{h}_{RA}}{\partial \theta_{DEC}} = \mathbf{0}$$

$$\frac{\partial \mathbf{h}_{RA}}{\partial \mathbf{b}_{RA}} = \mathbf{I}_3$$

$$\frac{\partial \mathbf{h}_{RA}}{\partial \mathbf{b}_{DEC}} = \mathbf{0}$$

From (2), $\mathbf{h}_{DEC}$ depends on θ_{RA} through the chained input $\mathbf{R}_{RA} \mathbf{T}_{RA} \mathbf{g}_p$, and on θ_{DEC} through $\mathbf{R}_{DEC}$ directly:

$$\frac{\partial \mathbf{h}_{DEC}}{\partial \theta_{RA}} = \mathbf{T}_{DEC} \mathbf{R}_{DEC} \mathbf{T}_{RA} \dot{\mathbf{R}}_{RA} \mathbf{g}_p$$

$$\frac{\partial \mathbf{h}_{DEC}}{\partial \theta_{DEC}} = \mathbf{T}_{DEC} \dot{\mathbf{R}}_{DEC} \mathbf{T}_{RA} \mathbf{R}_{RA} \mathbf{g}_p$$

Assembling these into the 6×8 Jacobian gives

$$\mathbf{C}_k = \begin{bmatrix} \frac{\partial \mathbf{h}_{RA}}{\partial \theta_{RA}} & \mathbf{0} & \mathbf{I}_3 & \mathbf{0} \\ \frac{\partial \mathbf{h}_{DEC}}{\partial \theta_{RA}} & \frac{\partial \mathbf{h}_{DEC}}{\partial \theta_{DEC}} & \mathbf{0} & \mathbf{I}_3 \end{bmatrix}, \quad (11)$$

B. Filter Equations

The EKF alternates between a measurement update, which incorporates the current accelerometer reading, and a time update, which propagates the state forward using the commanded step increment. The filter is initialized with $\hat{\mathbf{x}}_k^{(-)}$ from the initial accelerometer estimate and $\mathbf{P}_0^{(-)} = \text{blkdiag}(\sigma_{\theta,0}^2 \mathbf{I}_2, \sigma_{b,0}^2 \mathbf{I}_6)$, where $\sigma_{\theta,0}$ reflects the initial angle uncertainty from the static estimate and $\sigma_{b,0}$ reflects prior uncertainty on the accelerometer biases.

Measurement Update: At each sample k , $\mathbf{C}_k$ is evaluated at the current prior estimate $\hat{\mathbf{x}}_k^{(-)}$. The Kalman gain, posterior state, and covariance are then computed as:

$$\bar{\mathbf{K}}_k = \mathbf{P}_k^{(-)} \mathbf{C}_k^T (\mathbf{C}_k \mathbf{P}_k^{(-)} \mathbf{C}_k^T + \mathbf{R})^{-1} \quad (12)$$

$$\hat{\mathbf{x}}_k^{(+)} = \hat{\mathbf{x}}_k^{(-)} + \bar{\mathbf{K}}_k (\mathbf{y}_k - \mathbf{h}(\hat{\mathbf{x}}_k^{(-)})) \quad (13)$$

$$\mathbf{P}_k^{(+)} = (\mathbf{I} - \bar{\mathbf{K}}_k \mathbf{C}_k) \mathbf{P}_k^{(-)} \quad (14)$$

where $\mathbf{y}_k \in \mathbb{R}^6$ is the stacked accelerometer measurement vector and $\mathbf{R} = \sigma_{acc}^2 \mathbf{I}_6$.

Time Update: Because the process model is linear in the state, the step increment enters additively and the bias states are constant, $\Phi_k = \mathbf{I}_8$ and $\Gamma_w = \mathbf{I}_8$ for all k . The state is propagated to the next sample using the commanded angular increment $\Delta\theta_k$ from the step counter provided by the controller. Since the bias states are modeled as constant, only the angle states are driven by the input.

$$\hat{\mathbf{x}}_{k+1}^{(-)} = \hat{\mathbf{x}}_k^{(+)} + \begin{bmatrix} \Delta\theta_{RA,k} \\ \Delta\theta_{DEC,k} \\ \mathbf{0}_6 \end{bmatrix} \quad (15)$$

$$\mathbf{P}_{k+1}^{(-)} = \Phi_k \mathbf{P}_k^{(+)} \Phi_k^T + \Gamma_w \mathbf{Q} \Gamma_w^T = \mathbf{P}_k^{(+)} + \mathbf{Q} \quad (16)$$

Pass 2: Reduced-State Angle Filter: Once Pass 1 has converged, the bias estimates $\hat{\mathbf{b}}_{RA}$, $\hat{\mathbf{b}}_{DEC}$ are frozen and the filter is restarted as a 2-state system with $\hat{\mathbf{x}}_k = [\theta_{RA}, \theta_{DEC}]^T$ and $\mathbf{P}_0^{(-)}$ initialized from the top-left 2×2 block of $\mathbf{P}_N^{(+)}$ from Pass 1. The Jacobian reduces to the first two columns of (11).

$$\mathbf{C}_k^{(2)} = \begin{bmatrix} \frac{\partial \mathbf{h}_{RA}}{\partial \theta_{RA}} & \mathbf{0} \\ \frac{\partial \mathbf{h}_{DEC}}{\partial \theta_{RA}} & \frac{\partial \mathbf{h}_{DEC}}{\partial \theta_{DEC}} \end{bmatrix} \in \mathbb{R}^{6 \times 2}, \quad (17)$$

Pass 2 sweeps the measurement sequence in reverse, propagating the state backward using the same step-counter increments with reversed sign. This backward pass allows the angle uncertainty to converge toward the start of the trajectory, where the home position is extracted by finding the axis angle that satisfies the home condition defined in Section I. The specific extraction procedure is outside the scope of this work; the implementation is provided in the Appendix. The comparison of the estimated home position determined by the filter and the true home position established by the simulation is used as the metric for filter performance.

VI. FILTER PERFORMANCE

A. Pass 1: Nuisance Parameter Convergence

Figure 4 shows the bias estimates from a representative first pass. All six states converge toward their true values within approximately 2×10^4 samples or $\approx 140^\circ$ of rotation, with the true value falling within the $\pm 2\sigma$ bounds at steady state. The RA biases converge more rapidly than the DEC biases, consistent with the stronger observability of the RA bias states (Section IV). All six channels are within 0.4% of the expected noise floor ($\sigma_{acc} = 0.34$) m/s^2 and converged to the randomly sampled bias values. This confirms that the Pass 1 sufficiently estimates the biases with no systematic error in the calibrated measurement model.

B. Pass 2: Angle Estimate Convergence

Figure 5 shows the second pass angle estimates for a representative run. In both axes the EKF tracks the true trajectory closely throughout the sweep, with the true value remaining within the $\pm 2\sigma$ envelope at all samples. The right axis shows the 1σ uncertainty on a logarithmic scale, decreasing monotonically as the backward pass accumulates measurements toward

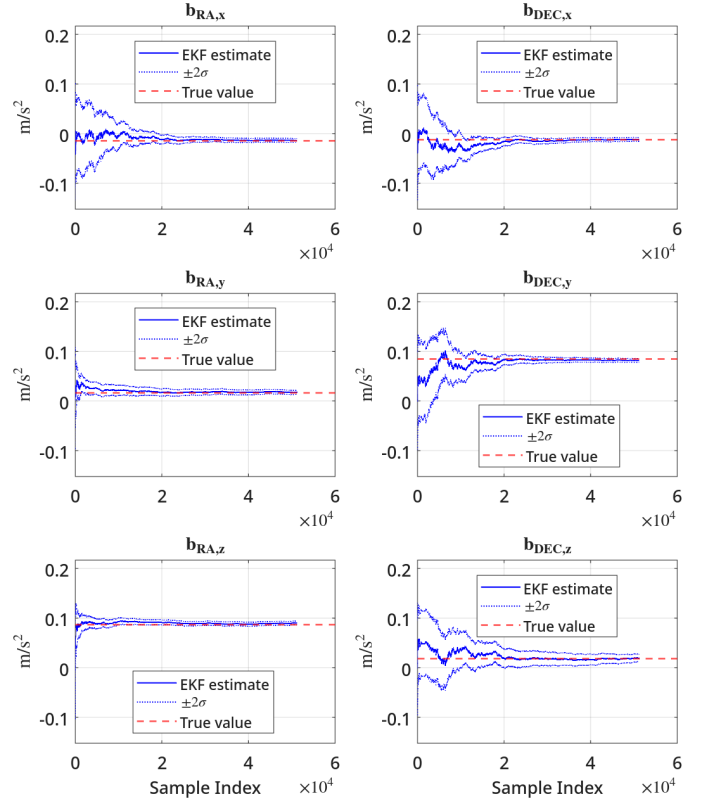


Fig. 4: Pass 1 bias estimates for all six accelerometer states. Blue solid: EKF estimate; blue dotted: $\pm 2\sigma$ bounds; red dashed: true value.

sample index zero, where the home position is extracted. The RA axis converges from approximately 0.014° to 0.010° , while DEC converges more aggressively from 0.030° to 0.010° . At the extraction point, a 1σ uncertainty 0.010° corresponds to approximately 7,000 microsteps at the stepper input shaft – well within the $\pm 25,600$ microstep homing budget.

C. Monte Carlo Analysis

The filter was evaluated over 100 Monte Carlo trials at two different site latitudes ($\lambda = [52^\circ, 40^\circ]$ for West Point, NY and Superior, CO respectively), each with independently sampled accelerometer biases and step-counter jitter, to assess homing reliability under representative uncertainty. A trial is considered a pass if the homing error on both axes falls within $\pm 25,600$ microsteps – corresponding to $\pm 180^\circ$ at the stepper input shaft.

TABLE II: Monte Carlo results over 100 trials at two site latitudes.

	$\lambda = 52$		$\lambda = 40$	
	RA	DEC	RA	DEC
Mean error (microsteps)	-1,807	382	-1,505	317
Std deviation (microsteps)	14,555	7,177	11,370	7,792
RMSE (microsteps)	14,594	7,151	11,413	7,759
Max abs error (microsteps)	38,948	17,392	28,115	19,867
Pass rate	92%	100%	99%	100%

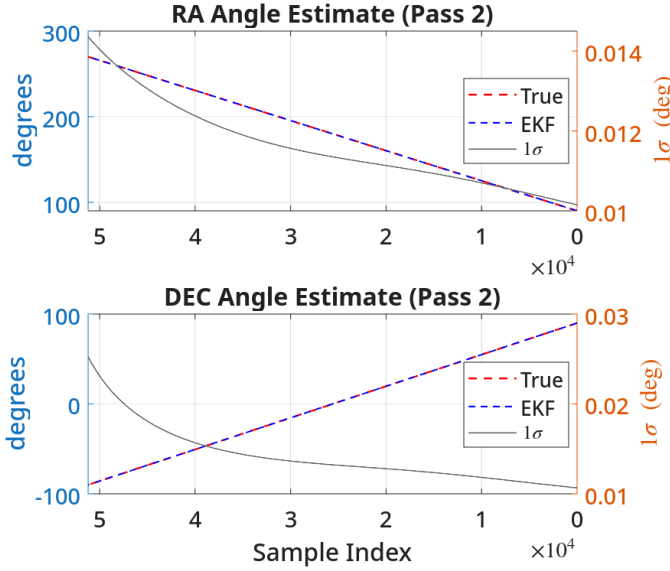


Fig. 5: Pass 2 angle estimates for RA (top) and DEC (bottom). Left axis: true trajectory (red dashed) and EKF estimate (blue dashed). Right axis: 1σ uncertainty envelope on a logarithmic scale. The x-axis is reversed to reflect the backward-pass direction, with the home extraction point at index zero.

Results are summarized in Table II and Figure 6. The DEC axis achieves a 100% pass rate at both latitudes. The RA axis passes in 92% of trials at $\lambda = 52^\circ$, improving to 99% at $\lambda = 40^\circ$, consistent with the latitude-dependent observability of θ_{RA} discussed in Section IV. The RA standard deviation decreases from 14,555 to 11,370 microsteps, and the maximum absolute error reduces from 38,948 to 28,115 microsteps – confirming improved performance at lower latitudes.

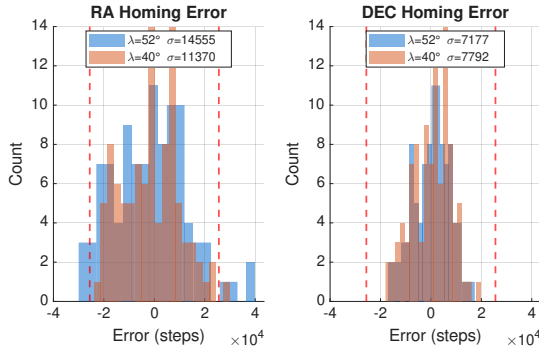


Fig. 6: Homing error distributions over 100 Monte Carlo trials for RA (left) and DEC (right). Red dashed lines indicate the $\pm 25,600$ step pass/fail boundary. At both site latitudes DEC achieves 100% pass rate, and the RA pass rate is improved at the lower latitude.

VII. DISCUSSION

Observability analysis confirmed that all eight states are uniquely recoverable along the planned trajectory, with the eigenvalue ratio $\lambda_8/\lambda_7 \approx 2.3$ between the DEC and RA angle

states predicting the factor-of-two difference in homing performance observed in simulation. The two-pass filter structure – forward bias estimation followed by a reduced 2-state backward angle filter – directly exploits this eigenvalue separation, dedicating the full measurement sequence to the least-observed states before recovering the angles with calibrated sensors.

Over 100 Monte Carlo trials at $\lambda = 52^\circ$, the filter achieved a 100% pass rate on the DEC axis and a 92% pass rate on the RA axis, with all failures attributable to the weaker observability of θ_{RA} . At $\lambda = 40^\circ$ the RA pass rate improved to 99%, consistent with the latitude dependence of RA observability. Post-calibration residuals on all six accelerometer channels were within 0.4% of the sensor noise floor, confirming that the bias estimates had converged prior to the angle estimation pass.

To mitigate the latitude dependence of RA observability, it is recommended that the homing slew be performed with the mount tilted to a fixed altitude angle of $\lambda = 40^\circ$ rather than in the polar-aligned location dependent configuration. Decoupling the homing procedure from the polar alignment ensures more consistent gravity projection onto the RA accelerometer axes across a range of site latitudes, potentially improving RA observability independent of geographic location.

A 99% pass rate is considered acceptable for the intended use case. Homing is required only once to establish the PEC table. Afterwards, the absolute step position can be retained in microcontroller memory across sessions, provided shutdown is graceful. The PEC table is only potentially invalidated by an uncontrolled power loss, which can be further mitigated by periodic EEPROM writes of the absolute position during normal operation. Write cycle longevity should be managed through standard wear-levelling or update-rate limiting strategies to prevent premature memory exhaustion. In the event of an uncontrolled power loss where EEPROM state cannot be guaranteed, the PEC table should be revalidated against a guide star or recreated in full prior to resuming precision imaging.

VIII. CONCLUSION

This paper presented a two-pass Extended Kalman Filter for sensorless homing of an equatorial telescope mount using only body-fixed accelerometers. The filter jointly estimates rotational position on both the Right Ascension and Declination axes alongside six accelerometer bias states, exploiting the deterministic relationship between axis angle and gravity projection during a controlled slew. Monte Carlo simulation demonstrates 99% homing reliability at mid latitudes, with performance limited by the latitude-dependent observability of the RA axis. The two-pass structure directly exploits the eigenvalue separation between angle states, and the method requires no additional sensors beyond the body-fixed accelerometers already present on the mount.

Future work will deploy the algorithm on the physical mount, replacing the simulated step counter with real stepper pulse timestamps, and will validate the proposed altitude-angle homing configuration experimentally.

IX. APPENDIX - MATLAB SIMULATION CODE

```

1 % ekf_telescope_run.m
2 % Runs the EKF on simulated telescope mount data.
3 % Pass 1: joint estimation of [theta_RA, theta_DEC, b_RA(3), b_DEC(3), phi_RA, phi_DEC]
4 % from accelerometer data alone. Step counter enters only as process model input.
5 % Pass 2: angle refinement with nuisance parameters frozen to Pass 1 estimates.
6
7 clear; clc; close all;
8 %% -----
9 % SIMULATION DATA
10 %% -----
11 %rng(42);
12 g = 9.81;
13 g_world = [0; 0; -g];
14
15 ra_start_deg = 90.0;
16 ra_end_deg = -90.0;
17 dec_start_deg = 90.0;
18 dec_end_deg = -90.0;
19 lat_deg = 52.0;
20
21 bias_ra = (rand(3,1) * 0.2) - 0.1;
22 bias_dec = (rand(3,1) * 0.2) - 0.1;
23
24 readingNoise = 0.34;
25 step_interval = 2500;
26 step_stdev = 1200;
27
28 stepper_steps_per_rev = 200;
29 stepper_microsteps = 256;
30 gear_ratio = 5000;
31 steps_per_deg = stepper_steps_per_rev * stepper_microsteps * gear_ratio / 360.0;
32
33 N = max(...
34     round(abs(ra_end_deg - ra_start_deg) * steps_per_deg / step_interval), ...
35     round(abs(dec_end_deg - dec_start_deg) * steps_per_deg / step_interval));
36
37 ra_sim_steps = linspace(ra_start_deg * steps_per_deg, ...
38     ra_start_deg * steps_per_deg + step_interval * N, N) ...
39     + randn(1,N) * step_stdev;
40
41 dec_sim_steps = linspace(dec_start_deg * steps_per_deg, ...
42     dec_start_deg * steps_per_deg - step_interval * N, N) ...
43     + randn(1,N) * step_stdev;
44
45 ra_sim_angles_true = ra_sim_steps / steps_per_deg;
46 dec_sim_angles_true = dec_sim_steps / steps_per_deg;
47
48 phi_lat = deg2rad(90 - lat_deg);
49 R_lat = [cos(phi_lat) 0 sin(phi_lat); 0 1 0; -sin(phi_lat) 0 cos(phi_lat)];
50 g_polar = R_lat * g_world;
51
52 T_RA = eye(3);
53 T_DEC = [-1 0 0; 0 0 -1; 0 1 0];
54
55 ra_sim_acc_x_true = zeros(1,N); ra_sim_acc_y_true = zeros(1,N); ra_sim_acc_z_true = zeros(1,N);
56 dec_sim_acc_x_true = zeros(1,N); dec_sim_acc_y_true = zeros(1,N); dec_sim_acc_z_true = zeros(1,N);
57
58 for i = 1:N
59     ra_t = deg2rad(ra_sim_angles_true(i));
60     dec_t = deg2rad(dec_sim_angles_true(i));
61     R_RA = [cos(ra_t) -sin(ra_t) 0; sin(ra_t) cos(ra_t) 0; 0 0 1];
62     R_DEC = [1 0 0; 0 cos(dec_t) -sin(dec_t); 0 sin(dec_t) cos(dec_t)];
63     g_ra = T_RA * R_RA * g_polar;
64     g_dec = T_DEC * R_DEC * g_ra;
65     ra_sim_acc_x_true(i) = g_ra(1); ra_sim_acc_y_true(i) = g_ra(2); ra_sim_acc_z_true(i) = g_ra(3);
66     dec_sim_acc_x_true(i) = g_dec(1); dec_sim_acc_y_true(i) = g_dec(2); dec_sim_acc_z_true(i) = g_dec(3);
67 end
68
69 % apply noise and bias to true positions to form noisy measurements
70 ra_sim_acc_x = ra_sim_acc_x_true + bias_ra(1) + randn(1,N)*readingNoise;
71 ra_sim_acc_y = ra_sim_acc_y_true + bias_ra(2) + randn(1,N)*readingNoise;
72 ra_sim_acc_z = ra_sim_acc_z_true + bias_ra(3) + randn(1,N)*readingNoise;

```

```

73 dec_sim_acc_x = dec_sim_acc_x_true + bias_dec(1) + randn(1,N)*readingNoise;
74 dec_sim_acc_y = dec_sim_acc_y_true + bias_dec(2) + randn(1,N)*readingNoise;
75 dec_sim_acc_z = dec_sim_acc_z_true + bias_dec(3) + randn(1,N)*readingNoise;
76
77 y = [ra_sim_acc_x; ra_sim_acc_y; ra_sim_acc_z;
78      dec_sim_acc_x; dec_sim_acc_y; dec_sim_acc_z];
79
80 %% -----
81 % FILTER PARAMETERS
82 % -----
83 R_meas = (readingNoise^2) * eye(6);
84 sigma_bias_Q = 1e-5; % small expected to be constant
85 sigma_phi_Q = 1e-5;
86 % The process noise on the angle states - we are provided a discrete value
87 sigma_angle_Q = (step_stddev / sqrt(step_interval) / steps_per_deg) * pi/180; % ~34 microstep
88
89 %% -----
90 % PASS 1: 8-state joint filter
91 % State: x = [theta_RA, theta_DEC, b_RA(3), b_DEC(3)]
92 % -----
93 fprintf('Running_Pass_1:_joint_angle+_nuisance_parameter_estimation...\n');
94
95 nx1 = 8;
96 % States: [theta_RA, theta_DEC, b_RA(3), b_DEC(3)]
97 xk = [deg2rad(ra_start_deg); ... % theta_RA
98       deg2rad(dec_start_deg); ... % theta_DEC
99       zeros(6,1)]; ... % biases (unknown)
100
101 Pk = blkdiag(...
102      diag([deg2rad(2)^2, deg2rad(2)^2]), ... % angle prior: +-5 deg 1-sigma
103      diag(repmat(0.05^2, 1, 6))); ... % bias prior
104
105 % Process noise: angles driven by step counter jitter;
106 Q_p1 = blkdiag(...
107      diag([sigma_angle_Q^2, sigma_angle_Q^2]), ...
108      diag(repmat(sigma_bias_Q^2, 1, 6))); ...
109
110 % initialize pass 1
111 x_post_p1 = zeros(nx1, N);
112 P_post_p1 = zeros(nx1, nx1, N);
113
114 for k = 1:N
115     % --- Measurement update ---
116     h_pred = h_func(xk(1), xk(2), xk(3:5), xk(6:8), g_polar, T_RA, T_DEC);
117     %Ck = num_jacobian(@(x) h_func(x(1), x(2), x(3:5), x(6:8), g_polar, T_RA, T_DEC), xk);
118     Ck = analytic_jacobian(xk(1), xk(2), g_polar, T_RA, T_DEC);
119
120     Sk = Ck * Pk * Ck' + R_meas;
121     Kk = Pk * Ck' / Sk;
122     xk = xk + Kk * (y(:,k) - h_pred);
123     Pk = (eye(nx1) - Kk * Ck)*Pk;
124
125     x_post_p1(:,k) = xk;
126     P_post_p1(:,:,k) = Pk;
127
128     % --- Time update: propagate angles using step counter increments ---
129     % Only the delta matters here, not the absolute step count.
130     if k < N
131         delta_ra = (ra_sim_steps(k+1) - ra_sim_steps(k)) / steps_per_deg * pi/180;
132         delta_dec = (dec_sim_steps(k+1) - dec_sim_steps(k)) / steps_per_deg * pi/180;
133         xk(1) = xk(1) + delta_ra;
134         xk(2) = xk(2) + delta_dec;
135     end
136     % both Gamma and Phi are the identity matrix
137     Pk = Pk + Q_p1;
138 end
139
140 % Extract calibration results from end of pass
141 b_RA_est = x_post_p1(3:5, end);
142 b_DEC_est = x_post_p1(6:8, end);
143
144 fprintf('Pass_1_complete.\n');
145 fprintf('b_RA: [%0.4f, %0.4f, %0.4f] m/s^2 (true: [%0.4f, %0.4f, %0.4f])\n', b_RA_est, bias_ra);
146 fprintf('b_DEC: [%0.4f, %0.4f, %0.4f] m/s^2 (true: [%0.4f, %0.4f, %0.4f])\n', b_DEC_est, bias_dec);

```



```

147 fprintf('theta_RA_final: %.4f_deg (true: %.4f_deg)\n', rad2deg(x_post_p1(1,end)), ra_sim_angles_true(
    end));
148 fprintf('theta_DEC_final: %.4f_deg (true: %.4f_deg)\n', rad2deg(x_post_p1(2,end)), dec_sim_angles_true(
    end));
149
150 [~, flag] = chol(P_post_p1(:, :, end));
151 fprintf('P_post_p1_PD_check: d_0=ok)\n', flag);
152
153 %% -----
154 % PASS 2: 2-state filter - estimate [theta_RA, theta_DEC]
155 % Nuisance parameters frozen to Pass 1 estimates
156 % -----
157 fprintf('\nRunning Pass 2: angle estimation with calibrated sensor...\n');
158
159 nx2 = 2;
160 xk = [x_post_p1(1, end); x_post_p1(2, end)];
161 Pk = P_post_p1(1:2, 1:2, end);
162 Q_p2 = diag([sigma_angle_Q^2, sigma_angle_Q^2]);
163
164 x_post_p2 = zeros(nx2, N);
165 P_post_p2 = zeros(nx2, nx2, N);
166
167 for k = N:-1:1
168     h_pred = h_func(xk(1), xk(2), b_RA_est, b_DEC_est, g_polar, T_RA, T_DEC);
169     %Ck = num_jacobian(@(x) h_func(x(1), x(2), b_RA_est, b_DEC_est, g_polar, T_RA, T_DEC), xk);
170     Ck = analytic_jacobian(xk(1), xk(2), g_polar, T_RA, T_DEC);
171     Ck = Ck(:, 1:2);
172
173     Sk = Ck * Pk * Ck' + R_meas;
174     Kk = Pk * Ck' / Sk;
175     xk = xk + Kk * (y(:, k) - h_pred);
176     Pk = (eye(nx2) - Kk * Ck) * Pk;
177
178     x_post_p2(:, k) = xk;
179     P_post_p2(:, :, k) = Pk;
180
181     if k > 1
182         xk(1) = xk(1) - (ra_sim_steps(k) - ra_sim_steps(k-1)) / steps_per_deg * pi/180;
183         xk(2) = xk(2) - (dec_sim_steps(k) - dec_sim_steps(k-1)) / steps_per_deg * pi/180;
184     end
185     % both Gamma and Phi are the identity matrix
186     Pk = Pk + Q_p2;
187 end
188
189 fprintf('Pass 2 complete.\n');
190 fprintf('P(1,1): %.4f_deg (start) -> %.4f_deg (end)\n', rad2deg(sqrt(P_post_p2(1,1,end))), rad2deg(sqrt(
    P_post_p2(1,1,1))));
191 fprintf('P(2,2): %.4f_deg (start) -> %.4f_deg (end)\n', rad2deg(sqrt(P_post_p2(2,2,end))), rad2deg(sqrt(
    P_post_p2(2,2,1))));
192
193 %% -----
194 % HOME POSITION EXTRACTION
195 % -----
196 theta_ra_now = x_post_p2(1, 1);
197 theta_dec_now = x_post_p2(2, 1);
198
199 % determine at what angle provides for an expected g=0.0 on the appropriate
200 % axis for each the RA and DEC axes
201 ra_home_est = theta_ra_now - deg2rad(ra_start_deg);
202 ra_home_angle = fzero(@(th) get_channel(h_func(th, theta_dec_now, b_RA_est, b_DEC_est, g_polar, T_RA, T_DEC
    ), 2), [ra_home_est - pi/2, ra_home_est + pi/2]);
203 dec_home_angle = fzero(@(th) get_channel(h_func(theta_ra_now, th, b_RA_est, b_DEC_est, g_polar, T_RA, T_DEC
    ), 6), theta_dec_now);
204
205 % convert back to the required microsteps to get to the home position
206 steps_to_home_ra = (ra_home_angle - theta_ra_now) * steps_per_deg * (180/pi);
207 steps_to_home_dec = (dec_home_angle - theta_dec_now) * steps_per_deg * (180/pi);
208
209 % the final EKF uncertainty propagates to the home position
210 sigma_ra_step = sqrt(P_post_p2(1,1,1)) * steps_per_deg * (180/pi);
211 sigma_dec_step = sqrt(P_post_p2(2,2,1)) * steps_per_deg * (180/pi);
212
213 % True home angles using true parameters
214 ra_home_angle_true = fzero(@(th) get_channel( ...

```

```

215     h_func(th, deg2rad(dec_sim_angles_true(1)), bias_ra, bias_dec, g_polar, T_RA, T_DEC), 2), ...
216     deg2rad(ra_sim_angles_true(1)));
217
218 dec_home_angle_true = fzero(@(th) get_channel( ...
219     h_func(deg2rad(ra_sim_angles_true(1)), th, bias_ra, bias_dec, g_polar, T_RA, T_DEC), 6), ...
220     deg2rad(dec_sim_angles_true(1)));
221
222 steps_to_home_ra_true = (rad2deg(ra_home_angle_true) - ra_sim_angles_true(1)) * steps_per_deg;
223 steps_to_home_dec_true = (rad2deg(dec_home_angle_true) - dec_sim_angles_true(1)) * steps_per_deg;
224
225 error_ra_steps = steps_to_home_ra - steps_to_home_ra_true;
226 error_dec_steps = steps_to_home_dec - steps_to_home_dec_true;
227
228 fprintf('\n---_Home_Position_---\n');
229 fprintf('RA: %f DEC: %f\n', steps_to_home_ra, steps_to_home_dec);
230 fprintf('Estimated_steps: %f\n', steps_to_home_ra, steps_to_home_dec);
231 fprintf('True_steps: %f\n', steps_to_home_ra_true, steps_to_home_dec_true);
232 fprintf('Error: %f\n', error_ra_steps, error_dec_steps);
233
234 %% -----
235 % PLOTS
236 % -----
237
238 % --- Pass 1: Nuisance parameter convergence ---
239 figure('Name','Pass_1:_Nuisance_Parameter_Convergence');
240 set(gcf,'Position',[100 100 900 900]);
241 labels = {'b_{RA,x}','b_{RA,y}','b_{RA,z}','b_{DEC,x}','b_{DEC,y}','b_{DEC,z}'};
242 true_vals = [bias_ra; bias_dec];
243
244 % Map state i to subplot position: RA(1-3) -> left col (1,3,5), DEC(4-6) -> right col (2,4,6)
245 subplot_pos = [1 3 5 2 4 6];
246
247 for i = 1:6
248     si = i + 2;
249     sig = squeeze(sqrt(P_post_pl(si, si, :)))';
250     subplot(3, 2, subplot_pos(i));
251     est = x_post_pl(si,:);
252     plot(est, 'b', 'LineWidth', 1.0); hold on;
253     plot(est + 2*sig, 'b:', 'LineWidth', 0.8);
254     plot(est - 2*sig, 'b:', 'LineWidth', 0.8);
255     yline(true_vals(i), 'r--', 'LineWidth', 1.2);
256     ylabel('m/s^2');
257     title(labels{i}, 'FontSize', 12);
258     legend('EKF_estimate', '2 \sigma', 'True_value', 'Location', 'best');
259     grid on;
260     if subplot_pos(i) >= 5, xlabel('Sample_Index'); end
261 end
262
263 % --- Pass 1: Angle estimates ---
264 figure('Name','Pass_1:_Joint_Angle_Estimates');
265 set(gcf,'Position',[100 100 900 500]);
266 sig_RA_pl = rad2deg(squeeze(sqrt(P_post_pl(1,1,:)))');
267 sig_DEC_pl = rad2deg(squeeze(sqrt(P_post_pl(2,2,:)))');
268
269 subplot(2,1,1);
270 plot(ra_sim_angles_true, 'r--', 'LineWidth', 1.2); hold on;
271 plot(rad2deg(x_post_pl(1,:)), 'b', 'LineWidth', 1.0);
272 plot(rad2deg(x_post_pl(1,:)) + 2*sig_RA_pl, 'b:', 'LineWidth', 0.8);
273 plot(rad2deg(x_post_pl(1,:)) - 2*sig_RA_pl, 'b:', 'LineWidth', 0.8);
274 ylabel('degrees'); title('RA_Angle_Estimate_(Pass_1_-joint)');
275 legend('True','EKF','2 \sigma','Location','best'); grid on;
276
277 subplot(2,1,2);
278 plot(dec_sim_angles_true, 'r--', 'LineWidth', 1.2); hold on;
279 plot(rad2deg(x_post_pl(2,:)), 'b', 'LineWidth', 1.0);
280 plot(rad2deg(x_post_pl(2,:)) + 2*sig_DEC_pl, 'b:', 'LineWidth', 0.8);
281 plot(rad2deg(x_post_pl(2,:)) - 2*sig_DEC_pl, 'b:', 'LineWidth', 0.8);
282 xlabel('Sample_Index'); ylabel('degrees'); title('DEC_Angle_Estimate_(Pass_1_-joint)');
283 legend('True','EKF','2 \sigma','Location','best'); grid on;
284
285 sig_RA = rad2deg(squeeze(sqrt(P_post_p2(1,1,:)))');
286 sig_DEC = rad2deg(squeeze(sqrt(P_post_p2(2,2,:)))');
287
288 figure('Name','Pass_2:_Angle_Estimates_and_Uncertainty');

```

```

289 set(gcf,'Position',[100 100 900 500]);
290
291 % RA
292 subplot(2,1,1);
293 yyaxis left;
294 plot(ra_sim_angles_true,'r--','LineWidth',1.2); hold on;
295 plot(rad2deg(x_post_p2(1,:)),'b','LineWidth',1.0);
296 ylabel('degrees');
297 yyaxis right;
298 semilogy(sig_RA,'Color',[0.4 0.4 0.4],'LineWidth',0.8);
299 ylabel('1\sigma_(deg)');
300 title('RA_Angle_Estimate_(Pass_2)');
301 legend('True','EKF','1\sigma','Location','best');
302 set(gca,'XDir','reverse');
303 xlim([0 N]);
304 grid on;
305
306 % DEC
307 subplot(2,1,2);
308 yyaxis left;
309 plot(dec_sim_angles_true,'r--','LineWidth',1.2); hold on;
310 plot(rad2deg(x_post_p2(2,:)),'b','LineWidth',1.0);
311 ylabel('degrees');
312 yyaxis right;
313 semilogy(sig_DEC,'Color',[0.4 0.4 0.4],'LineWidth',0.8);
314 ylabel('1\sigma_(deg)');
315 xlabel('Sample_Index');
316 title('DEC_Angle_Estimate_(Pass_2)');
317 legend('True','EKF','1\sigma','Location','best');
318 set(gca,'XDir','reverse');
319 xlim([0 N]);
320 grid on;
321
322 % --- Residuals after Pass 1 calibration ---
323 residuals = zeros(6, N);
324 for k = 1:N
325     h_cal = h_func(x_post_p1(1,k), x_post_p1(2,k), b_RA_est, b_DEC_est, g_polar, T_RA, T_DEC);
326     residuals(:,k) = y(:,k) - h_cal;
327 end
328 fprintf('\nResidual_std_per_channel_(should_be_~%.3f_m/s^2):\n', readingNoise);
329 fprintf('RA: [%.4f, %.4f, %.4f]\n', std(residuals(1,:)), std(residuals(2,:)), std(residuals(3,:)));
330 fprintf('DEC: [%.4f, %.4f, %.4f]\n', std(residuals(4,:)), std(residuals(5,:)), std(residuals(6,:)));
331
332 %% -----
333 % LOCAL FUNCTIONS
334 % -----
335 function h = h_func(theta_RA, theta_DEC, b_RA, b_DEC, g_polar, T_RA, T_DEC)
336     % form the rotation matrices
337     c_RA = cos(theta_RA);
338     s_RA = sin(theta_RA);
339     R_RA = [c_RA -s_RA 0; s_RA c_RA 0; 0 0 1];
340
341     c_DEC = cos(theta_DEC);
342     s_DEC = sin(theta_DEC);
343     R_DEC = [1 0 0; 0 c_DEC -s_DEC; 0 s_DEC c_DEC];
344
345     % determine the expected accelerometer measurements: eqns 3,4
346     g_ra = T_RA * R_RA * g_polar;
347     g_dec = T_DEC * R_DEC * g_ra;
348     h = [g_ra + b_RA; g_dec + b_DEC];
349 end
350
351 function C = analytic_jacobian(theta_RA, theta_DEC, g_polar, T_RA, T_DEC)
352     c_RA = cos(theta_RA); s_RA = sin(theta_RA);
353     c_DEC = cos(theta_DEC); s_DEC = sin(theta_DEC);
354
355     R_RA = [c_RA -s_RA 0; s_RA c_RA 0; 0 0 1];
356     R_DEC = [1 0 0; 0 c_DEC -s_DEC; 0 s_DEC c_DEC];
357
358     dRRA = [-s_RA -c_RA 0; c_RA -s_RA 0; 0 0 0];
359     dRDEC = [0 0 0; 0 -s_DEC -c_DEC; 0 c_DEC -s_DEC];
360
361     dh1_dtRA = T_RA * dRRA * g_polar;
362     dh2_dtRA = T_DEC * R_DEC * T_RA * dRRA * g_polar;

```

```

363     dh2_dtDEC = T_DEC * drDEC * T_RA * R_RA * g_polar;
364
365     C = [dh1_dtRA,    zeros(3,1), eye(3),    zeros(3,3);
366          dh2_dtRA,    dh2_dtDEC, zeros(3,3), eye(3)    ];
367 end
368
369 function J = num_jacobian(f, x)
370     nx = length(x);
371     f0 = f(x);
372     ny = length(f0);
373     J = zeros(ny, nx);
374     for j = 1:nx
375         e = zeros(nx,1);
376         eps = max(abs(x(j)),1)*1e-6;
377         e(j) = eps;
378         J(:,j) = (f(x+e) - f0) / eps;
379     end
380 end
381
382 function v = get_channel(h, idx)
383     v = h(idx);
384 end
385
386 %% -----
387 % OBSERVABILITY ANALYSIS
388 % -----
389 % Stack Jacobians into observability matrix
390 O = zeros(6*N, nx1);
391 for k = 1:N
392     Ck = analytic_jacobian(x_post_p1(1,k), x_post_p1(2,k), g_polar, T_RA, T_DEC);
393     O((k-1)*6+1 : k*6, :) = Ck;
394 end
395
396 fprintf('\n---_Observability_Analysis_---\n');
397 fprintf('O_size:_%d_x_%d\n', size(O,1), size(O,2));
398 fprintf('O_rank:_%d/_%d\n', rank(O), nx1);
399
400 if rank(O) == nx1
401     fprintf('System_is_observable_(O_has_full_column_rank)\n');
402 else
403     fprintf('System_is_NOT_observable_(%d_unobservable_directions)\n', nx1 - rank(O));
404 end
405
406 % W = O'*O for eigenvalue analysis of observability quality
407 W = O' * O;
408
409 % State order must match x = [theta_RA, theta_DEC, b_RA(3), b_DEC(3)]
410 state_labels = {'\theta_{RA}', '\theta_{DEC}', ...
411                 'b_{RA,x}', 'b_{RA,y}', 'b_{RA,z}', ...
412                 'b_{DEC,x}', 'b_{DEC,y}', 'b_{DEC,z}'};
413
414 [V, D] = eig(W);
415 eigenvalues = diag(D);
416 [eigenvalues, sort_idx] = sort(eigenvalues, 'ascend');
417 V = V(:, sort_idx);
418
419 fprintf('\nCondition_number:_.2e\n', cond(W));
420 fprintf('\nEigenvalues_(ascending)_-_small_=_poorly_observed:\n');
421 for i = 1:nx1
422     [~, dom] = max(abs(V(:,i)));
423     fprintf('_\lambda_=%%.4e_(dominant:_%s)\n', i, eigenvalues(i), state_labels{dom});
424 end

```